

INMOGEST

(DAS) Documento de Arquitectura de Software

Versión 3.0

Integrantes:

Williams Contreras

Horacio Navarrete

Docente: Gerardo Galán

Asignatura: Ingeniería de Software

Sección: 001D

Identificación de Documento

Identificación	Informe de Arquitectura de Software
Proyecto	INMOGEST
Versión	3.0
Documento mantenido por	Williams Contreras / Horacio Navarrete
Fecha de última revisión	25-05-2026
Fecha de próxima revisión	26-05-2026
Documento aprobado por	Williams Contreras
Fecha de última aprobación	25-05-2026

Historial de Revisiones

Fecha	Versión	Descripción	Autor
27-03-2026	0.1	Redacción inicial del documento y planteamiento de objetivos.	Williams - Horacio
30-03-2026	0.2	Desarrollo de planificación y elaboración de Carta Gantt.	Williams - Horacio
03-04-2026	0.	Corrección de ámbito del sistema y replanteamiento de objetivos.	Williams - Horacio
07-04-2026	0.4	Análisis preliminar y corrección del documento.	Williams - Horacio
08-05-2026	2.0	Incorporación de diagramas UML del Modelo 4+1, estándares de calidad ISO/IEC 25010 y sección de Estilos y Patrones Arquitectónicos.	Williams - Horacio
25-05-2026	3.0	Incorporación de plantillas de control de pruebas y versiones del software.	Williams - Horacio

Tabla de Contenidos

1. Introducción

1.1 Contexto del Problema | 1.2 Propósito | 1.3 Ámbito del Sistema
1.4 Definiciones, Acrónimos y Abreviaciones | 1.5 Resumen Ejecutivo | 1.6 Arquitectura del Sistema (Vistas 4+1)

2. Visión del Sistema

2.1 Descripción General | 2.2 Objetivos | 2.3 Requerimientos Funcionales
2.4 Requerimientos No Funcionales | 2.5 Supuestos y Dependencias

3. Estilos y Patrones Arquitectónicos

3.1 Estilo Arquitectónico Adoptado | 3.2 Justificación | 3.3 Patrones de Diseño Aplicados

4. Modelamiento UML — Vistas 4+1

4.1 Vista de Escenarios (Casos de Uso) | 4.2 Vista Lógica (Clases)
4.3 Vista de Desarrollo (Componentes) | 4.4 Vista de Proceso (Actividad) | 4.5 Vista Física (Despliegue)

5. Estándares de Calidad del Software

6. Anexos

6.1 Tarjetas del Tablero Kanban
6.2 Tablero Kanban

1. INTRODUCCIÓN

1.1 Contexto del Problema

El contexto del problema surge a partir de las deficiencias detectadas en la administración del edificio Mirador, un edificio de condominios ubicado en el centro de la ciudad. Actualmente, la administración presenta falencias significativas en la atención a residentes, en la gestión de procesos administrativos, en la transparencia del manejo de activos y en la relación Administrador-Residente.

Esta situación ha generado desconfianza entre los copropietarios, conflictos vecinales, posibles fugas de dinero y dificultades en la correcta ejecución de proyectos de mantención. La operación manual y el uso de canales informales (bitácoras físicas, grupos de WhatsApp) agravan la falta de trazabilidad y control.

1.2 Propósito

El propósito de este documento es describir la arquitectura de software del sistema INMOGEST, estableciendo las decisiones técnicas, estructurales y de diseño que guiarán su desarrollo. Se busca automatizar tareas administrativas y proveer herramientas que fortalezcan la confianza entre administradores y residentes, situando la transparencia como pilar fundamental en la gestión de solicitudes, pagos y documentación del edificio.

Este documento sirve como referencia para el equipo de desarrollo, el docente evaluador y cualquier stakeholder que requiera comprender el sistema desde una perspectiva arquitectónica.

1.3 Ámbito del Sistema

El ámbito del sistema corresponde a la gestión administrativa integral de condominios y los procesos cotidianos del edificio Mirador. El sistema permitirá a los residentes acceder a documentación de su unidad (comprobantes de pago y estados de cuenta), realizar pagos en línea y comunicarse con la administración. Por su parte, la administración podrá compartir documentación sobre proyectos, desglosar el uso de activos y gestionar solicitudes, facilitando la transición desde un método análogo a uno digital.

El sistema NO abarca la gestión de otros edificios fuera del edificio Mirador en esta primera versión, ni la integración con sistemas contables externos más allá de la pasarela de pagos.

1.4 Definiciones, Acrónimos y Abreviaciones

Acrónimo	Descripción
Equipo TI	Encargados de implementación del sistema y funcionalidad en óptimas condiciones, enfocados en la satisfacción del usuario final.
SGI	Sistema de Gestión Integral
ADMIN	Usuario que ocupa el rol de administrador en el edificio y sistema.
PROP	Usuario registrado como propietario de una de las propiedades del edificio.
ARD	Usuario con contrato de arriendo de una de las propiedades del edificio.
CONS	Usuario que desempeña el trabajo de conserje dentro del edificio.
Unidad	Unidad de vivienda individual dentro del edificio.

Acrónimo	Descripción
Request	Solicitud de alguna incidencia ocurrida en alguna de las unidades.
MVC	Modelo-Vista-Controlador, patrón de arquitectura de software.
UML	Unified Modeling Language, lenguaje unificado de modelado.
RF	Requerimiento Funcional.
RNF	Requerimiento No Funcional.
DAS	Documento de Arquitectura de Software.

1.5 Resumen Ejecutivo

El proyecto INMOGEST surge como respuesta a las deficiencias detectadas en la administración del edificio Mirador, las cuales incluyen falta de transparencia, ineficiencia en la gestión de procesos administrativos y dificultades en la relación entre administradores y residentes. Estas problemáticas han generado desconfianza, conflictos vecinales y posibles pérdidas económicas.

El propósito principal del sistema es modernizar y optimizar la gestión administrativa mediante la digitalización y automatización de procesos clave, permitiendo una comunicación más fluida, acceso oportuno a la información y mayor control sobre los recursos del edificio.

INMOGEST es un Sistema de Gestión Integral (SGI) que permitirá a los residentes acceder fácilmente a documentación relevante (pagos, solicitudes), mientras que la administración podrá gestionar proyectos, transparentar el uso de recursos y mantener un registro ordenado de todas las operaciones.

Desde el punto de vista arquitectónico, el sistema se implementará como una **aplicación web monolítica bajo el patrón MVC** (Modelo-Vista-Controlador), estructurada en tres capas: presentación, lógica de negocio y acceso a datos. La arquitectura ha sido modelada siguiendo el estándar de vistas 4+1 de Philippe Kruchten, cubriendo los diagramas de Casos de Uso, Clases, Componentes, Actividad y Despliegue.

La implementación busca generar impactos positivos medibles: aumento en la liquidez del edificio, mayor inversión en mejoras, incremento en la satisfacción de los residentes y reducción de la morosidad.

1.6 Arquitectura del Sistema — Vistas 4+1

La arquitectura de INMOGEST se describe mediante el modelo de vistas 4+1 propuesto por Philippe Kruchten, que permite representar el sistema desde distintas perspectivas para distintos stakeholders:

a) Vista de Escenarios (Casos de Uso)

Define los principales flujos de interacción entre los actores del sistema y sus funcionalidades. Los escenarios principales incluyen:

- Un residente consulta su estado de cuenta de gastos comunes.
- Un residente realiza el pago en línea de su deuda mediante pasarela de pago.
- El administrador genera el cálculo mensual de gastos comunes y emite cobros.
- El sistema identifica automáticamente a los residentes morosos y genera alertas.
- Un residente registra una solicitud de mantención o una queja.
- El conserje registra incidencias relacionadas con el edificio.
- El administrador genera informes financieros y los exporta.

b) Vista Lógica (Diagrama de Clases)

El sistema se estructura en los siguientes módulos lógicos: Gestión de Usuarios, Finanzas (gastos comunes, cobros, pagos, morosos), Notificaciones, Solicitudes y Quejas, Reportes, y Seguridad (autenticación y autorización por roles).

c) Vista de Desarrollo (Diagrama de Componentes)

Se adopta el patrón MVC con tres capas claramente definidas: **Capa de Presentación (View)** — interfaces diferenciadas por rol; **Capa de Lógica de Negocio (Controller)** — gestión de reglas de negocio, cálculos y flujos; **Capa de Acceso a Datos (Model)** — repositorios de entidades del dominio conectados a la base de datos relacional.

d) Vista de Proceso (Diagrama de Actividad)

El sistema contempla procesamiento en tiempo real para pagos, generación de tareas programadas (emisión mensual de cobros, informes periódicos) y gestión de eventos asincrónicos (envío de notificaciones automáticas ante vencimientos o nuevas solicitudes).

e) Vista Física (Diagrama de Despliegue)

El sistema se despliega como una aplicación web monolítica en un proveedor de hosting en la nube (AWS / Google Cloud / Azure), con una base de datos relacional separada. El acceso se realiza mediante navegadores web modernos desde cualquier dispositivo con conexión a internet.

2. VISIÓN DEL SISTEMA

INMOGEST es un sistema de software de gestión integral diseñado para condominios y edificios residenciales que buscan optimizar y automatizar sus tareas cotidianas. Facilita una gestión eficiente tanto para administradores como para residentes, mejorando la comunicación y los procesos internos, diferenciándose por su enfoque en la transparencia, eficiencia y satisfacción del usuario final.

2.1 Descripción General del Sistema

El sistema está orientado a la eficiencia operativa y a la transparencia de la gestión. La organización de activos y la administración financiera del edificio dejan de ser una carga manual para convertirse en una herramienta digital que mejora los espacios comunes y la relación entre residentes y administración. INMOGEST centraliza en una sola plataforma web todas las operaciones que actualmente se realizan de forma fragmentada: cobros, pagos, solicitudes, reportes e incidencias.

2.2 Objetivos del Sistema

Objetivo General:

- Optimizar la administración general del edificio Mirador, incluyendo la atención de los residentes y sus necesidades.
- Modernizar el sistema de gestión residencial mediante la implementación de un sistema informático accesible y transparente.

Objetivos Específicos:

- Mejorar el control financiero respecto a las deudas y gastos comunes de los residentes.
- Potenciar la comunicación entre los residentes y el personal del edificio.
- Manejar de forma sistemática las solicitudes de emergencia y mantenimiento de los residentes.
- Optimizar la gestión y respuesta del personal del edificio frente a incidencias y solicitudes.
- Proveer un panel de transparencia que permita a los residentes conocer el uso de los recursos del edificio.

2.3 Requerimientos Funcionales

Código	Requisito	Descripción
RF-01	Registro de usuarios	Permite crear cuentas de usuarios del sistema con validación de datos.
RF-02	Gestión de perfiles	Permite administrar y actualizar información de usuarios.
RF-03	Autenticación de usuarios	Controla el acceso mediante credenciales para mantener la seguridad del sistema.
RF-04	Autorización por roles	Restringe accesos y funcionalidades según perfil (ADMIN, PROP, ARD, CONS).
RF-05	Interfaz de residentes	Permite al residente consultar su estado de cuenta, realizar pagos y enviar solicitudes.
RF-06	Interfaz de administrador	Panel central para gestión de cobros, residentes, reportes y configuración del sistema.

Código	Requisito	Descripción
RF-07	Interfaz de conserje	Herramienta operativa para registro de incidencias y atención de solicitudes.
RF-08	Cálculo de gastos comunes	Automatiza el cálculo mensual de gastos comunes según parámetros definidos.
RF-09	Emisión de cobros	Genera y distribuye cobros mensuales a cada unidad.
RF-10	Gestión de pagos	Permite registrar y procesar pagos en línea mediante pasarela externa.
RF-11	Identificación de morosos	Detecta automáticamente a residentes con deudas vencidas y genera alertas.
RF-12	Notificaciones automáticas	Envía avisos del sistema ante vencimientos, nuevas solicitudes o respuestas.
RF-13	Solicitudes de mantención	Gestiona requerimientos de mantención por parte de residentes.
RF-14	Gestión de quejas	Administra y da seguimiento a reclamos de residentes.
RF-15	Generación de informes	Genera reportes financieros y operativos configurables.
RF-16	Panel de transparencia	Muestra información financiera del edificio de forma accesible a los residentes.
RF-17	Gestión de personal	Administra datos y roles de los trabajadores del edificio.
RF-18	Historial de transacciones	Guarda y permite consultar el historial completo de movimientos financieros.
RF-19	Gestión de incidencias	Controla el registro y seguimiento de incidentes del edificio.
RF-20	Exportación de datos	Exporta información en formatos estándar (PDF, Excel/CSV).

2.4 Requerimientos No Funcionales

Código	Requisito	Descripción
RNF-01	Seguridad de datos	Protección de información mediante cifrado en tránsito (HTTPS/TLS) y en reposo. Cumplimiento con la Ley de Protección de Datos Personales de Chile.
RNF-02	Disponibilidad	Garantía de acceso continuo al sistema con una disponibilidad mínima del 99% (SLA).
RNF-03	Usabilidad	Interfaz intuitiva, accesible para usuarios no técnicos (residentes y personal del edificio).
RNF-04	Responsividad	Adaptación de la interfaz a distintos dispositivos: computadores, tablets y smartphones.
RNF-05	Escalabilidad	El sistema debe soportar el crecimiento en cantidad de usuarios y unidades sin degradar el rendimiento.
RNF-06	Integridad de datos	Mantenimiento de la consistencia y exactitud de datos financieros y de usuarios.
RNF-07	Tiempo de respuesta	Las operaciones comunes deben completarse en menos de 3 segundos bajo carga normal.
RNF-08	Mantenibilidad	Código estructurado bajo MVC que facilite cambios, correcciones y extensiones futuras.
RNF-09	Autenticación y autorización	Control de accesos basado en roles con tokens JWT y sesiones seguras.
RNF-10	Interoperabilidad	Integración con servicios externos: pasarela de pagos (Webpay/Transbank, MercadoPago o Fintoc) y servicios de correo electrónico para notificaciones.

2.5 Supuestos y Dependencias

Supuestos:

- **Acceso a Internet y Dispositivos:** Se asume que la gran mayoría de los residentes y el personal del edificio cuentan con smartphones, tablets o computadores con acceso a internet para utilizar el sistema.
- **Disposición de Datos Iniciales:** Se asume que la administración del edificio Mirador entregará los datos históricos (lista de residentes, deudas, inventario) en un formato estructurado (Excel o CSV) para la carga inicial en la base de datos.
- **Adopción del Sistema:** Se asume que al menos un porcentaje mayoritario de los residentes utilizará la plataforma, condición necesaria para cumplir las métricas de éxito (reducción de morosidad).
- **Transición de Canal de Comunicación:** Se asume que la administración establecerá INMOGEST como el canal único y oficial para gestión de solicitudes, quejas y avisos, descontinuoando gradualmente el uso de canales informales.

Dependencias:

- **Pasarela de Pagos:** El sistema dependerá de un proveedor externo de servicios de pago (Webpay de Transbank, MercadoPago o Fintoc) para el procesamiento seguro de pagos en línea.

- **Infraestructura de Hosting:** El sistema depende de un proveedor de alojamiento en la nube (AWS, Google Cloud, Azure o VPS) para el despliegue de la aplicación y la base de datos.
- **Marco Legal:** Dependencia del cumplimiento de la Ley de Copropiedad Inmobiliaria vigente en Chile y de la Ley de Protección de Datos Personales (Ley 19.628 y su futura actualización).
- **Compatibilidad de Navegadores:** El correcto funcionamiento de las interfaces depende de navegadores web modernos (Google Chrome, Safari, Microsoft Edge).

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1 Estilo Arquitectónico Adoptado

INMOGEST adopta una **arquitectura monolítica en capas**, implementada bajo el patrón **MVC (Modelo-Vista-Controlador)**. El sistema se despliega como una única unidad de software con tres capas claramente diferenciadas:

- **Capa de Presentación (View):** Interfaces de usuario web adaptadas a cada rol (ADMIN, PROP/ARD, CONS). Responsable de la presentación de datos y la recepción de entradas del usuario.
- **Capa de Lógica de Negocio (Controller):** Procesamiento de reglas de negocio, flujos de trabajo, cálculos de gastos comunes, validaciones y coordinación entre la vista y el modelo.
- **Capa de Acceso a Datos (Model):** Entidades del dominio y repositorios responsables de la persistencia y consulta de datos en la base de datos relacional.

3.2 Justificación del Estilo según el Contexto del Sistema

- **Simplicidad y velocidad de desarrollo:** El equipo de desarrollo es reducido (2 integrantes) y el sistema tiene un alcance acotado a un edificio. Una arquitectura monolítica reduce la complejidad operacional inicial al no requerir la gestión de múltiples servicios independientes.
- **Cohesión del dominio:** Todos los módulos del sistema comparten el mismo contexto de negocio (la administración de un edificio residencial). No existe justificación suficiente para separar estos módulos en microservicios en esta etapa.
- **Mantenibilidad adecuada para el contexto:** El patrón MVC separa claramente las responsabilidades, facilitando que el equipo TI pueda mantener y extender el sistema sin que los cambios en la interfaz afecten la lógica de negocio.
- **Cumplimiento de requerimientos no funcionales:** La arquitectura en capas favorece la mantenibilidad (RNF-08), la seguridad (RNF-01) al centralizar el control de acceso, y la escalabilidad vertical (RNF-05).
- **Contexto académico y de primera versión:** El monolito MVC provee una base sólida y comprensible, que puede evolucionar hacia SOA o microservicios en versiones futuras si el edificio requiere escalar a múltiples condominios.

3.3 Patrones de Diseño Aplicados

- **Patrón MVC (Modelo-Vista-Controlador):** Patrón central de la arquitectura. Separa la interfaz de usuario, la lógica de negocio y el acceso a datos en componentes independientes y colaborativos. Aplicado globalmente en el sistema.
- **Patrón Repositorio (Repository Pattern):** Abstrae el acceso a la base de datos detrás de interfaces de repositorio por cada entidad del dominio (UsuarioRepository, PagoRepository, SolicitudRepository, etc.).
- **Patrón Observador (Observer / Event-Driven):** Utilizado en el módulo de notificaciones. Cuando ocurre un evento relevante (vencimiento de cobro, nueva solicitud, moroso identificado), el sistema publica el evento y el módulo de notificaciones lo consume de forma asíncrona.
- **Patrón Estrategia (Strategy):** Aplicado en el módulo de pagos para permitir intercambiar la pasarela de pagos (Transbank, MercadoPago, Fintoc) sin modificar la lógica central del sistema.
- **Patrón Fachada (Facade):** Utilizado para simplificar la interacción con servicios externos (pasarela de pagos, servicio de correo electrónico), exponiendo una interfaz simple hacia los controladores del sistema.

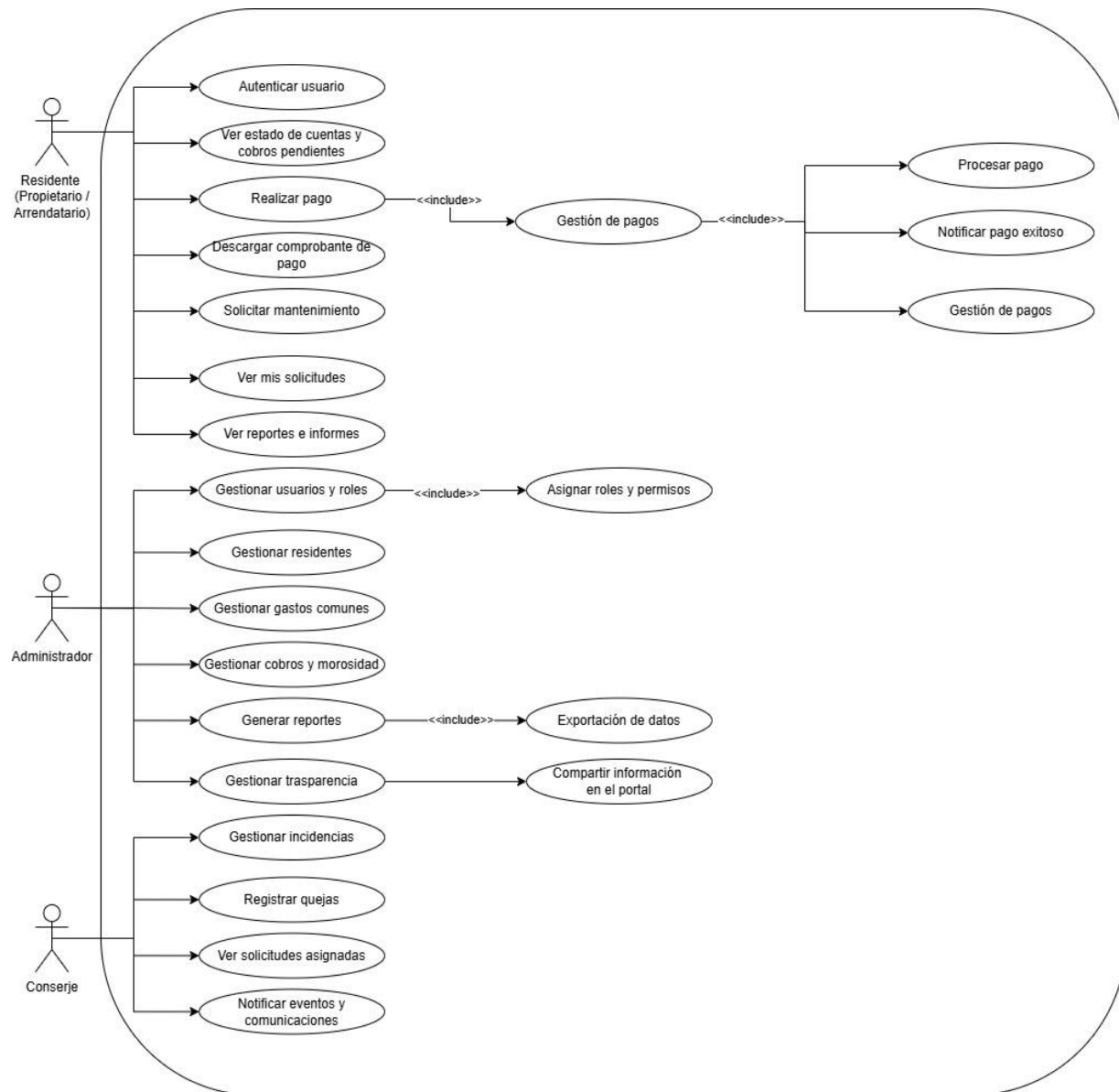
4. MODELAMIENTO UML — VISTAS 4+1

4.1 Vista de Escenarios — Diagrama de Casos de Uso

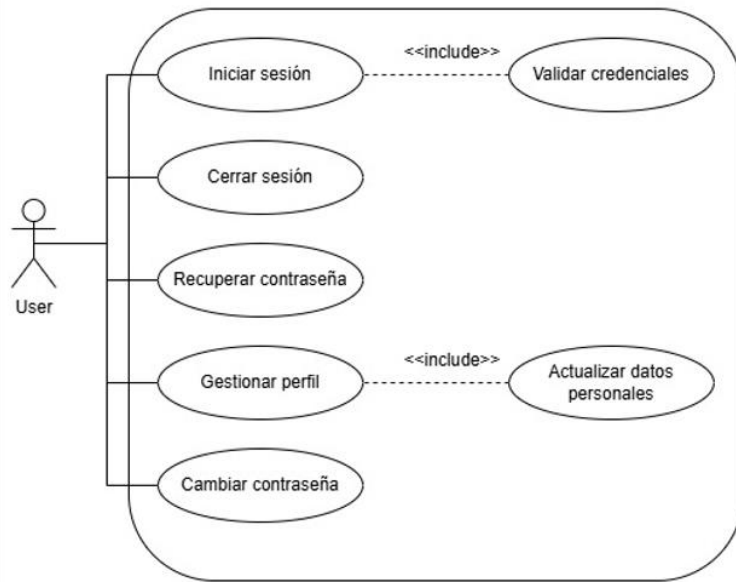
Propósito: Captura los requisitos funcionales desde la perspectiva del usuario. Permite identificar actores, sus interacciones con el sistema y los casos de uso principales.

Actores: Arrendatario (ARD), Propietario (PROP), Administrador (ADMIN), Conserje (CONS).

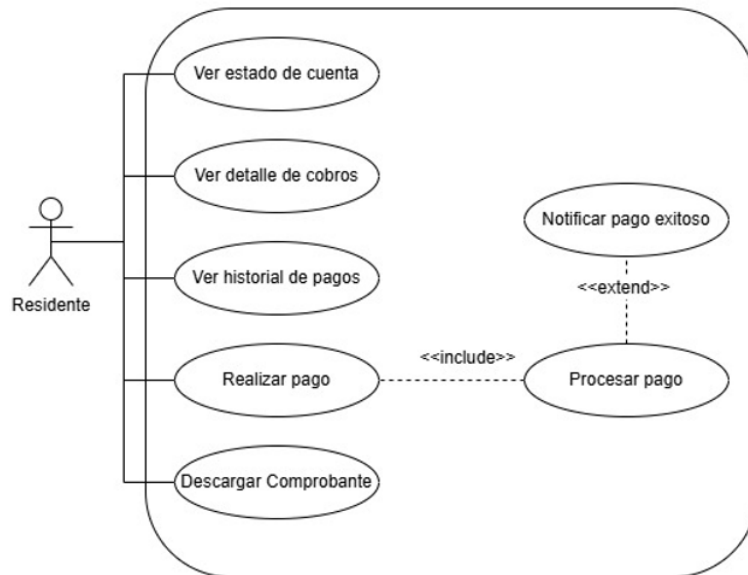
Casos de uso principales: Crear/Registrar Usuarios, Autenticar Usuario (JWT), Autorización por Roles, Realización de Pago, Solicitar Mantención, Generación de Informes, Gestión de Incidencias.



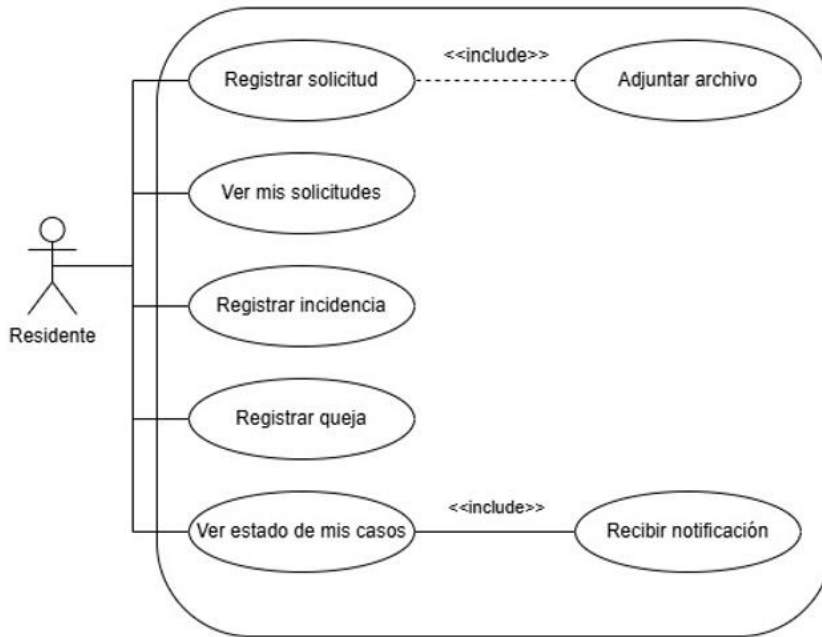
Autenticación y Gestión de Usuario



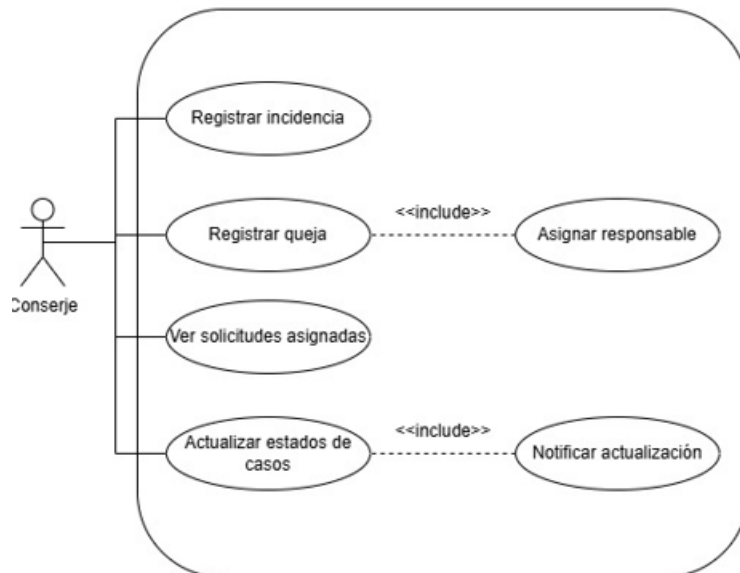
Gestión de Gastos Comunes y Cobros

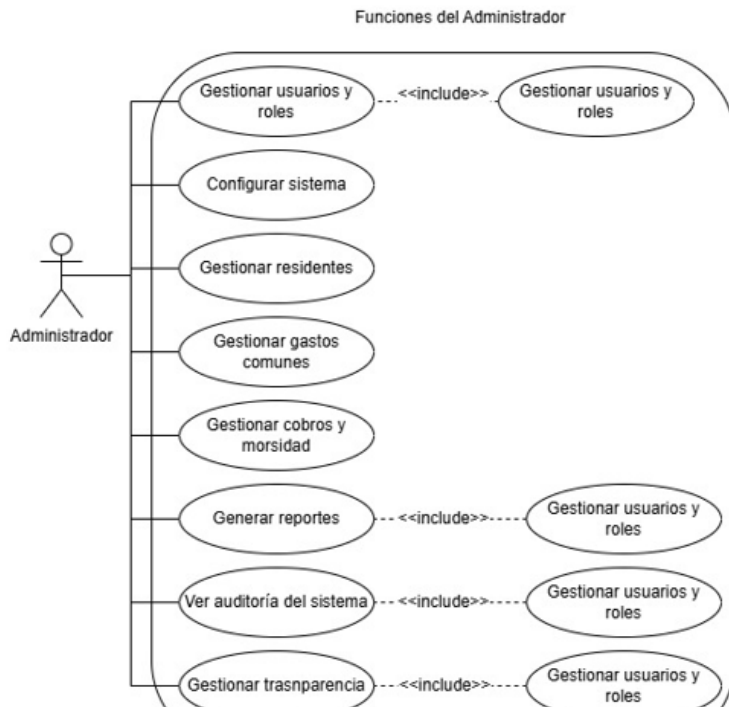


Gestión de Solicitudes, Incidencias y Quejas



Funciones del conserje



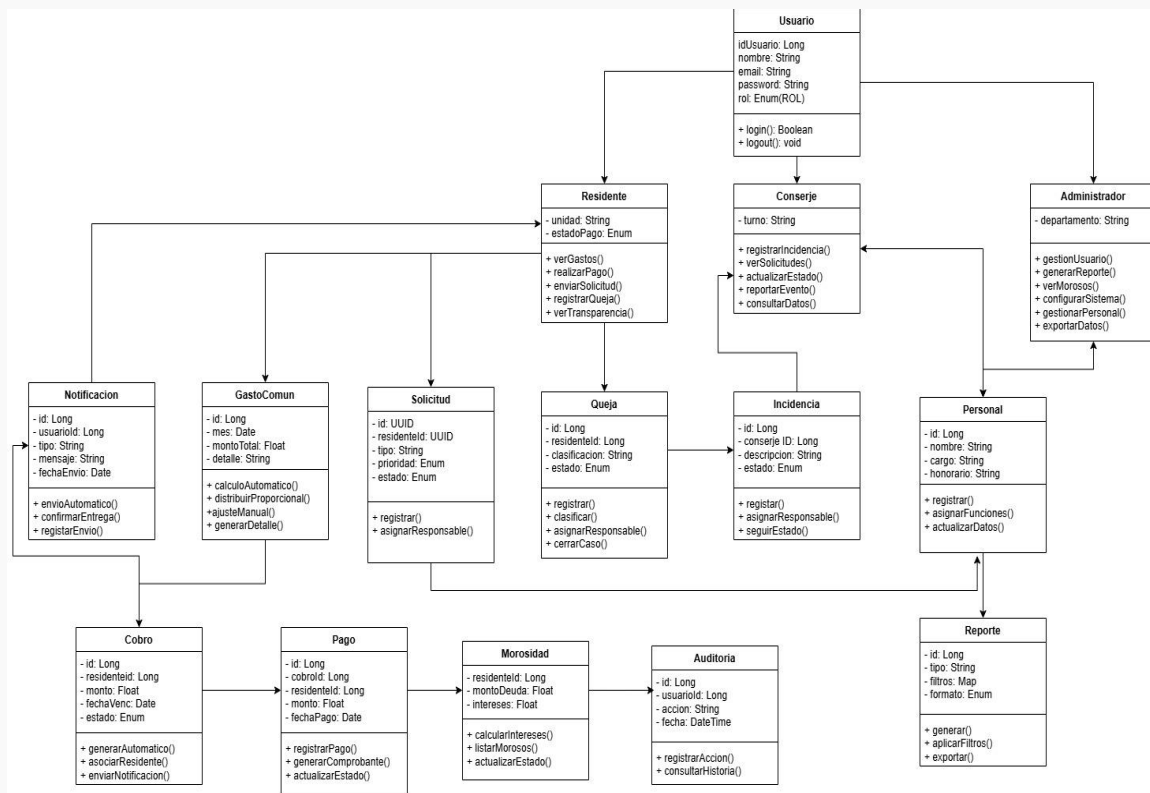


4.2 Vista Lógica — Diagrama de Clases

Propósito: Describe la estructura estática del sistema, mostrando las clases del dominio, sus atributos, métodos y relaciones.

Clases principales: Usuario (base con subclases Administrador, Propietario, Arrendatario, Conserje), Unidad, GastoComun, Cobro, Pago, Solicitud, Incidencia, Notificacion.

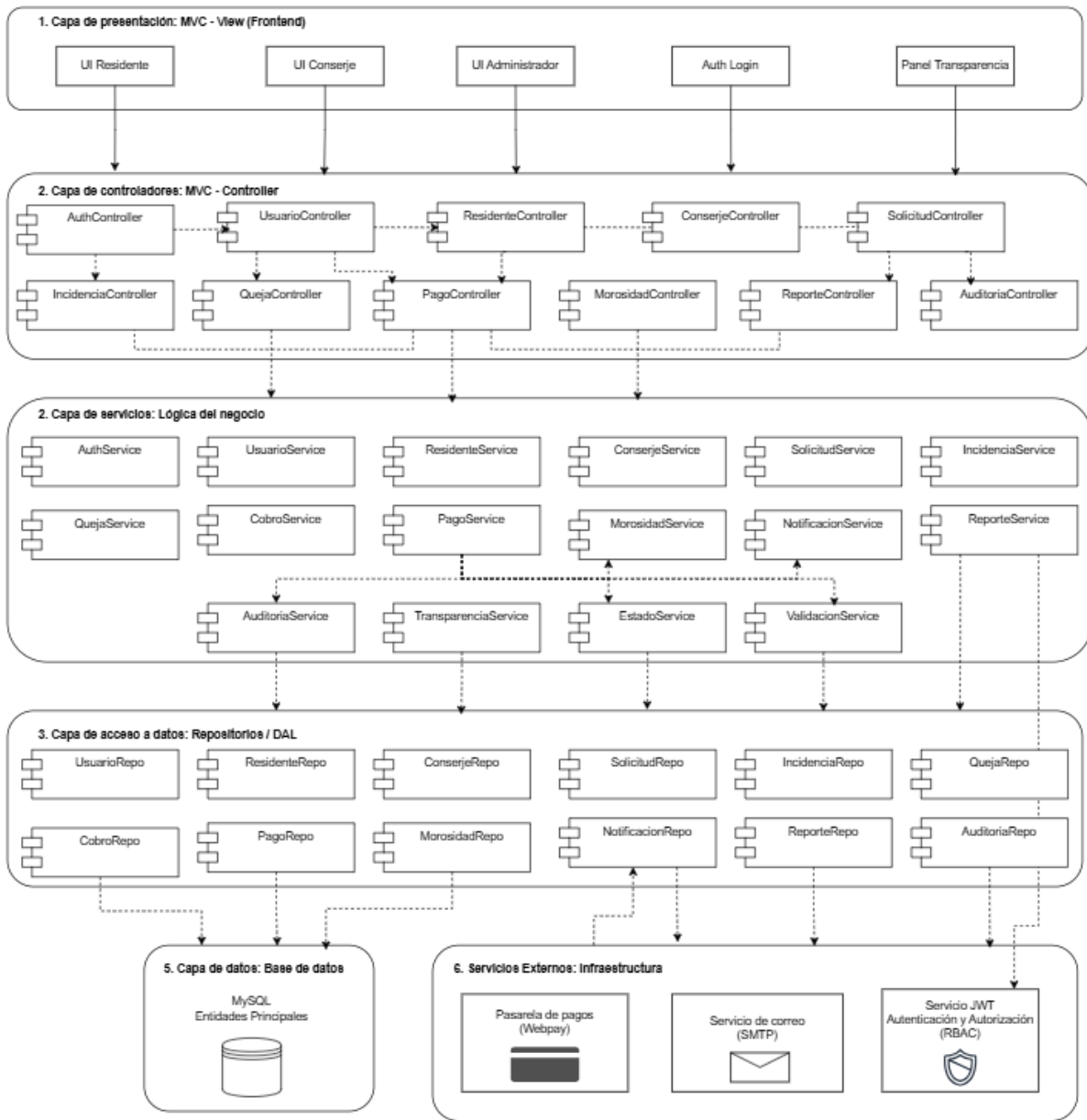
Relaciones clave: Un Usuario puede ser ADMIN, PROP, ARD o CONS (herencia). Una Unidad tiene uno o más Cobros. Un Cobro genera cero o un Pago. Un Cobro vencido dispara una Notificacion automática.



4.3 Vista de Desarrollo — Diagrama de Componentes

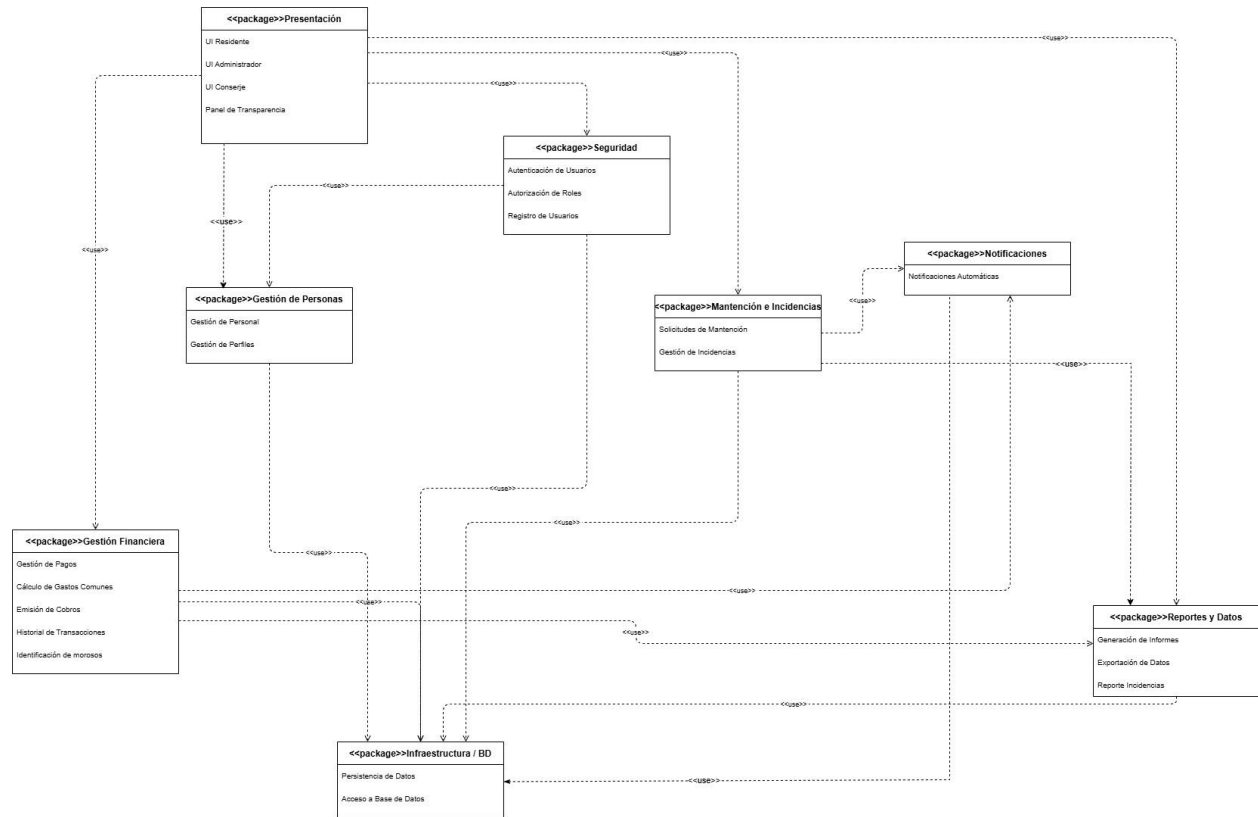
Propósito: Muestra la organización del código en componentes de software reutilizables, sus interfaces y dependencias.

Componentes principales: Frontend (UI Residente, UI Administrador, UI Conserje), Backend (AuthController, UsuarioController, FinanzasController, PagoController, SolicitudController, IncidenciaController, NotificacionController, ReporteController), Capa de Datos (Repositorios + ORM/DAL), Servicios Externos (Pasarela de Pagos, Servicio de Correo), Base de Datos relacional (PostgreSQL/MySQL).



4.4 Vista de Desarrollo — Diagrama de Paquetes

Propósito: Organizar y estructurar los distintos elementos de un sistema de software en grupos lógicos llamados paquetes. Este diagrama permite representar la división modular del sistema, facilitando la comprensión de cómo se agrupan las clases, componentes u otros elementos relacionados según su funcionalidad o responsabilidad.

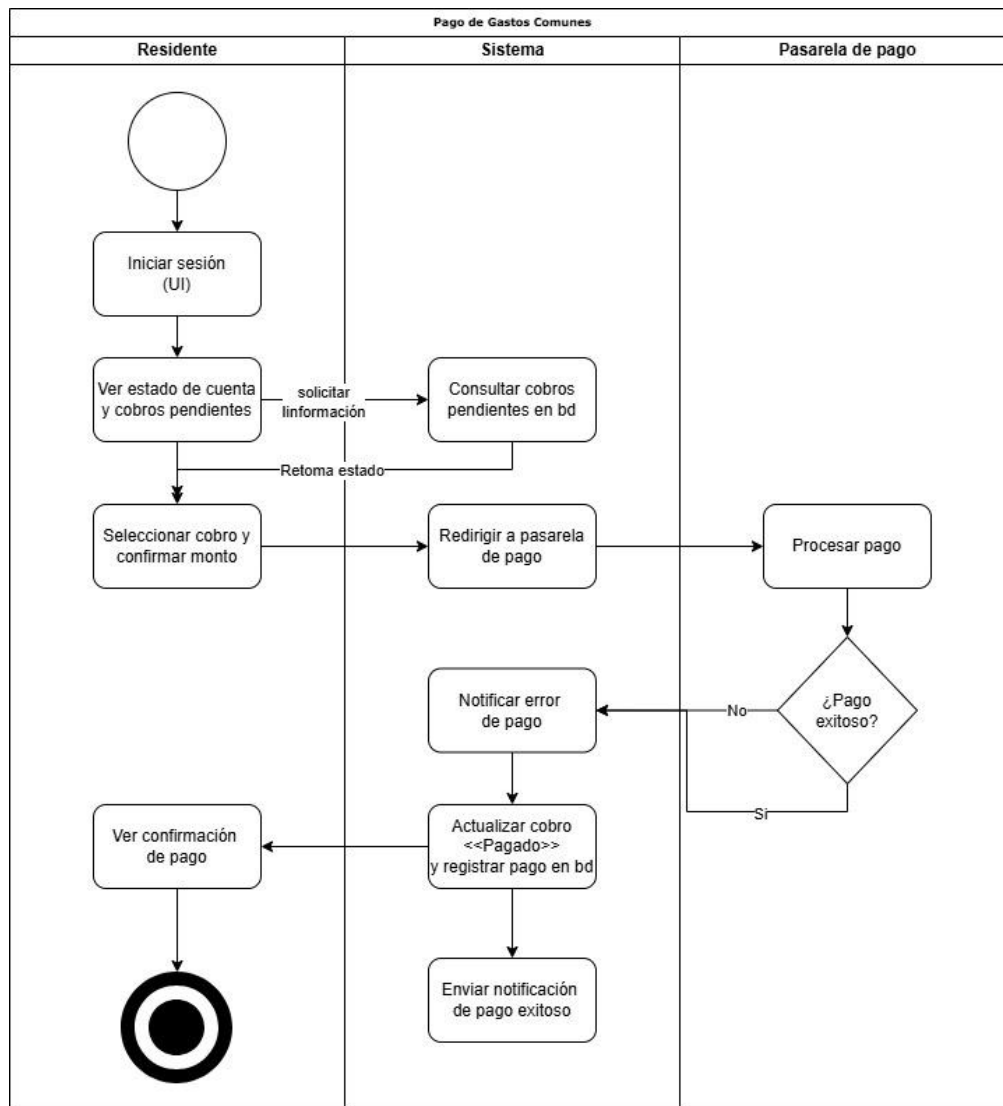


4.5 Vista de Proceso — Diagrama de Actividad

Propósito: Describe el comportamiento dinámico del sistema en tiempo de ejecución, mostrando flujos de trabajo, concurrencia e interacción entre procesos.

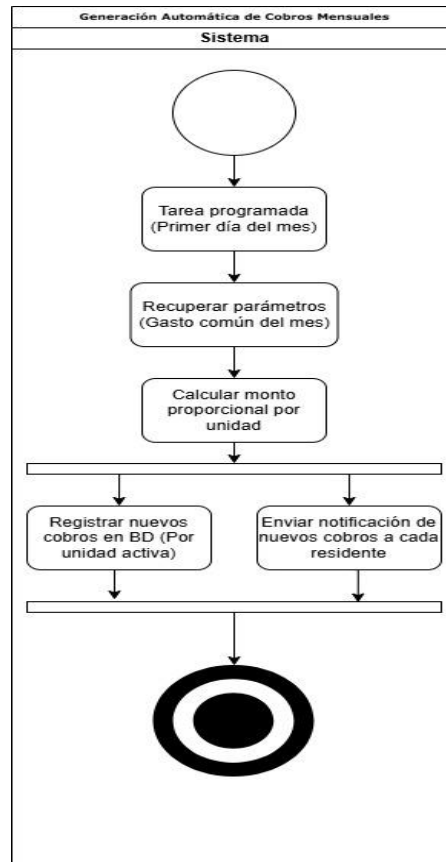
Flujo 1: Pago de Gastos Comunes por Residente

1. Residente inicia sesión (autenticación JWT).
2. Residente accede a su estado de cuenta.
3. Sistema consulta cobros pendientes en la base de datos.
4. Residente selecciona el cobro a pagar y confirma el monto.
5. Sistema redirige a la pasarela de pagos externa.
6. Pasarela procesa el pago y retorna estado (éxito/fracaso).
7. Si éxito: Sistema actualiza el Cobro a "pagado", registra el Pago y envía notificación.
8. Si fracaso: Sistema notifica al residente e invita a reintentar.



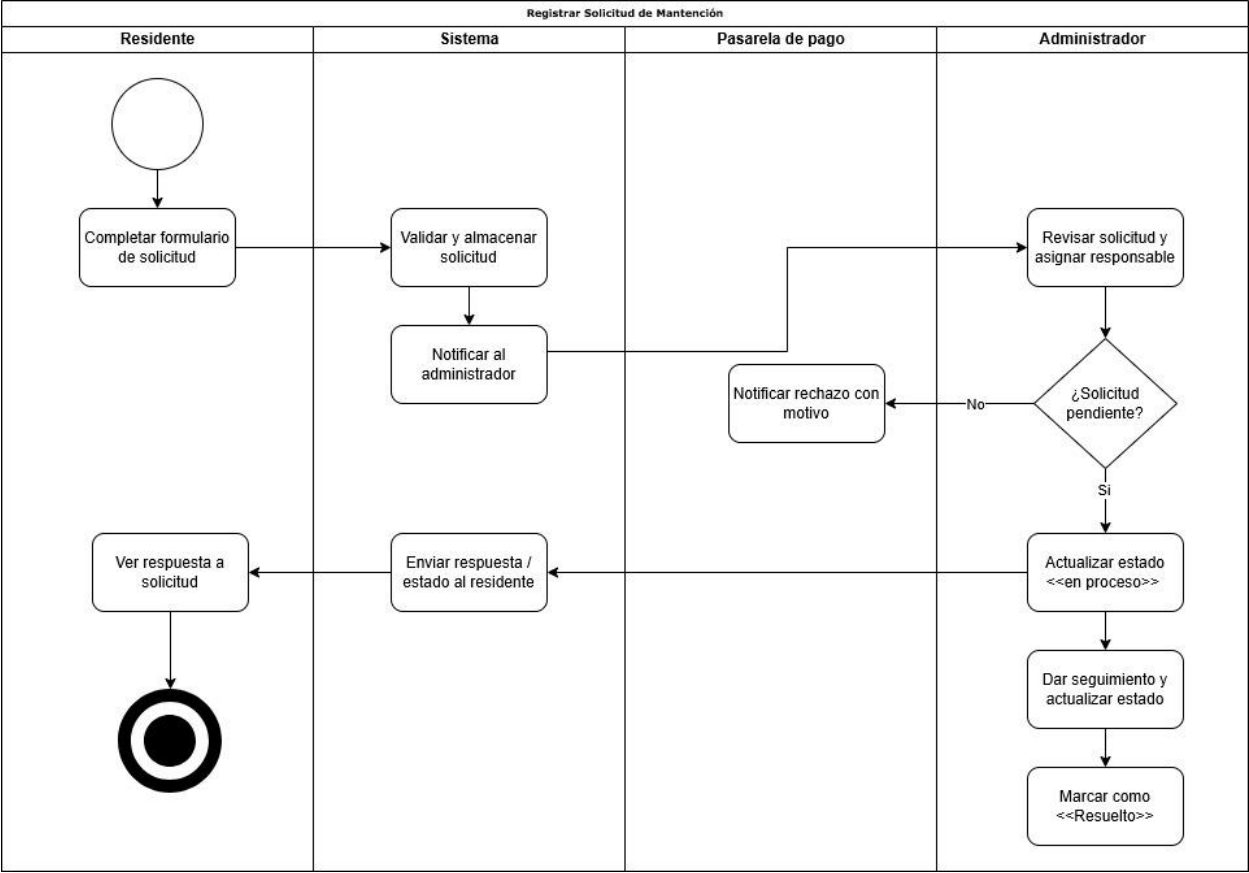
Flujo 2: Generación Automática de Cobros Mensuales (tarea programada)

1. Tarea programada se activa el primer día del mes.
2. Sistema recupera todos los parámetros del gasto común del mes anterior.
3. Sistema calcula el monto proporcional por unidad.
4. Sistema emite un Cobro para cada unidad activa.
5. Sistema envía notificación a cada residente informando el nuevo cobro.



Flujo 3: Registro de Solicitud de Mantenimiento por Residente

1. Residente inicia sesión.
2. Residente completa formulario de solicitud (tipo, descripción, adjuntos opcionales).
3. Sistema valida y almacena la solicitud.
4. Sistema notifica al Administrador.
5. Administrador revisa y responde la solicitud, actualizando su estado.
6. Sistema notifica al Residente con la respuesta.

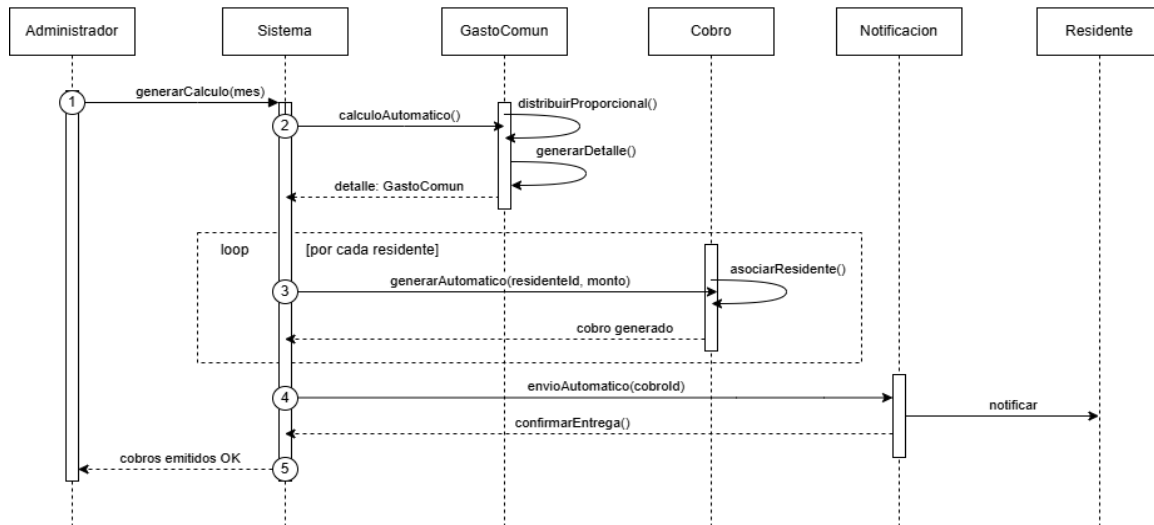


4.6 Vista de Procesos - Diagrama de secuencias

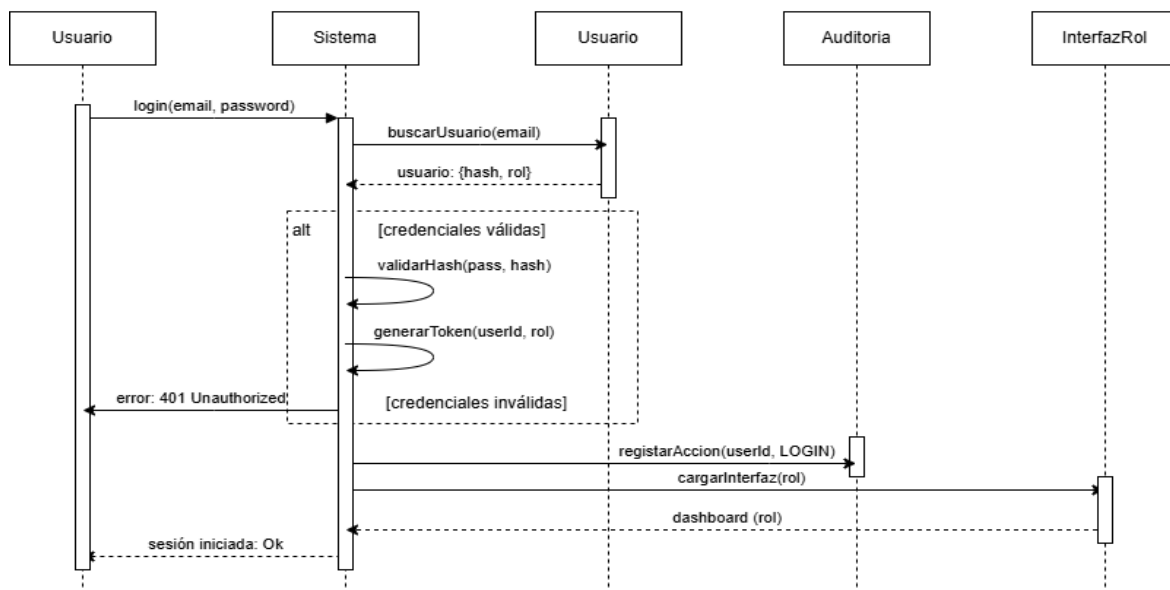
Propósito: El diagrama de secuencia permite visualizar la interacción temporal entre actores, sistema, base de datos y servicios externos durante un proceso.

Este diagrama ayuda a comprender el intercambio de mensajes y el orden en que ocurren las operaciones del sistema.

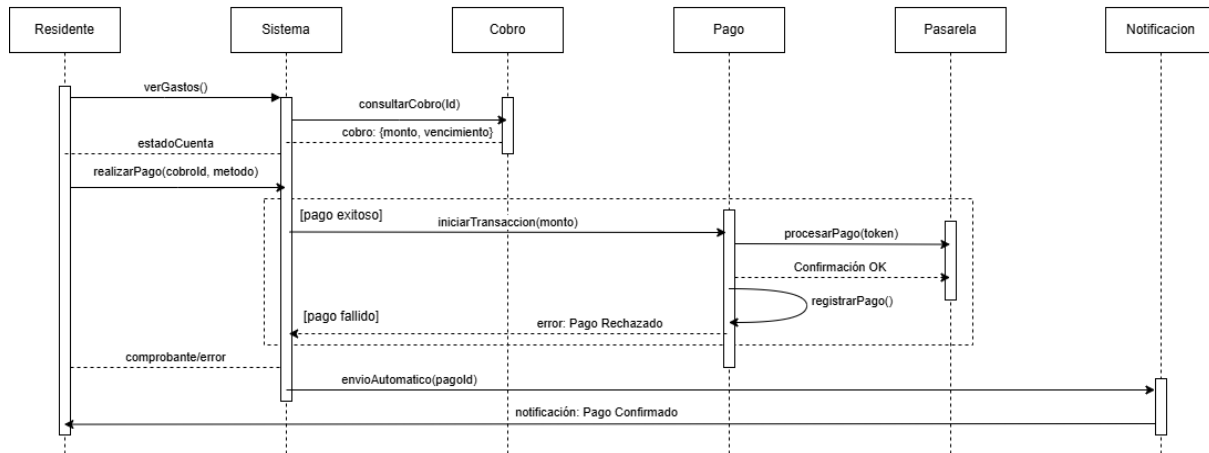
Cálculo de Gastos Comunes y Emisión de Cobros



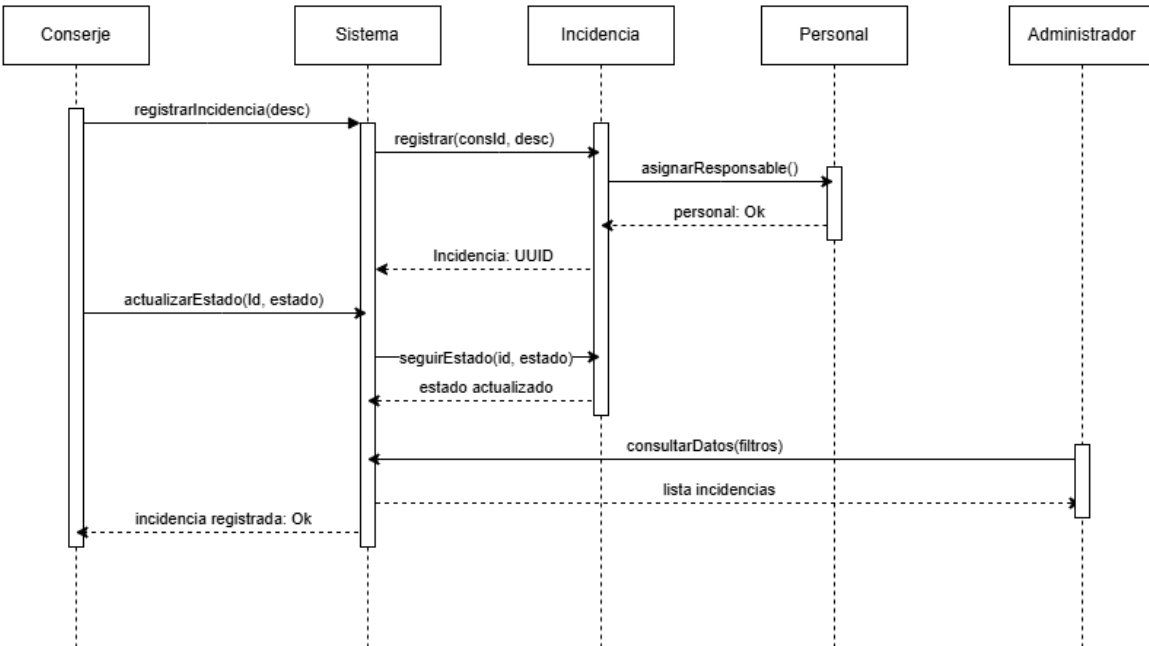
Autenticación de Usuarios y Autorización por Roles



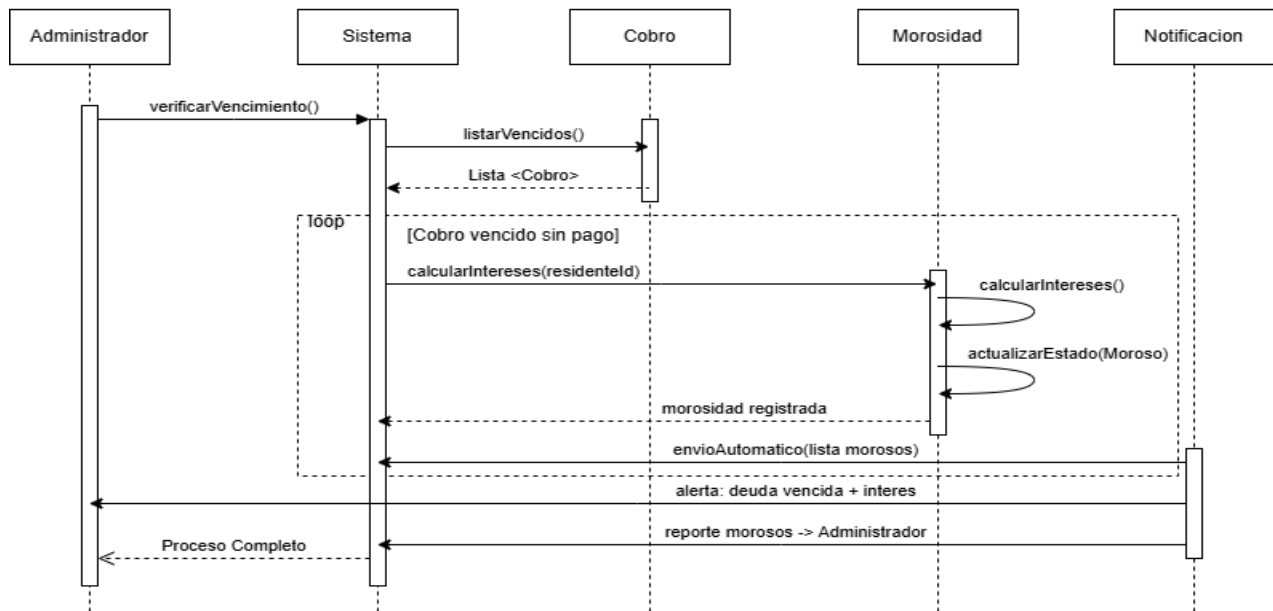
Gestión de Pagos



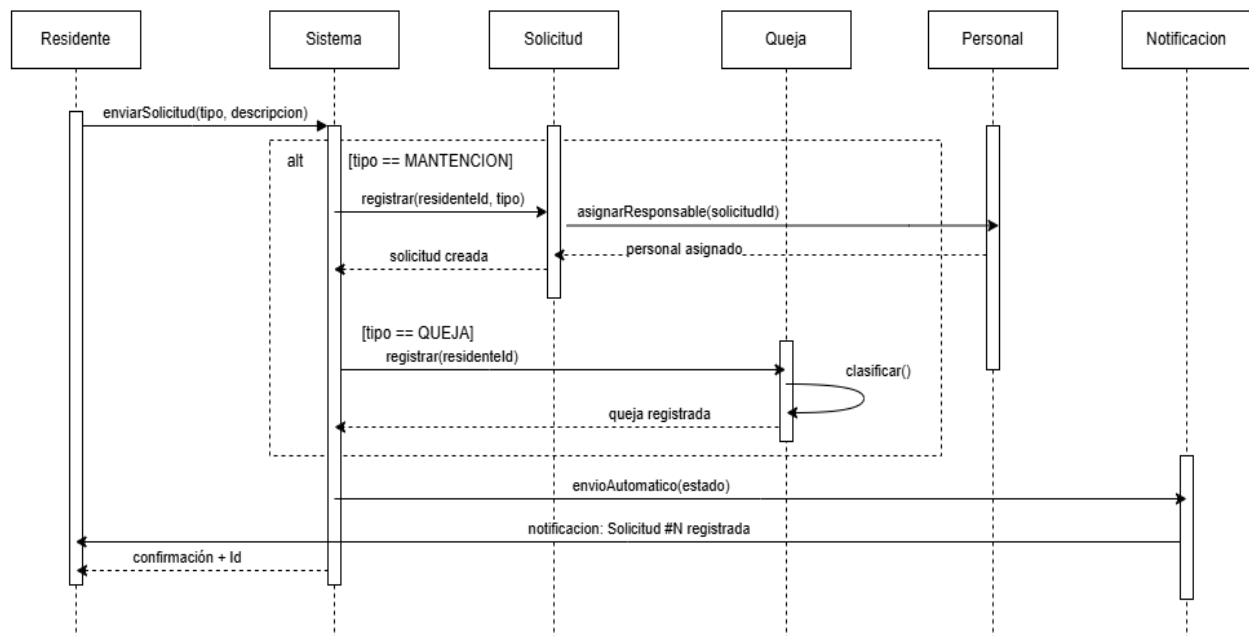
Registro y Seguimiento de Incidencias



Identificación de Morosos y Notificaciones Automáticas



Registro de Solicitud de Mantención y Gestión de Queja



4.7 Vista Física — Diagrama de Despliegue

Propósito: Describe la distribución del software en la infraestructura física y de red. Muestra cómo los componentes de software son asignados a nodos de hardware o de cómputo en la nube.

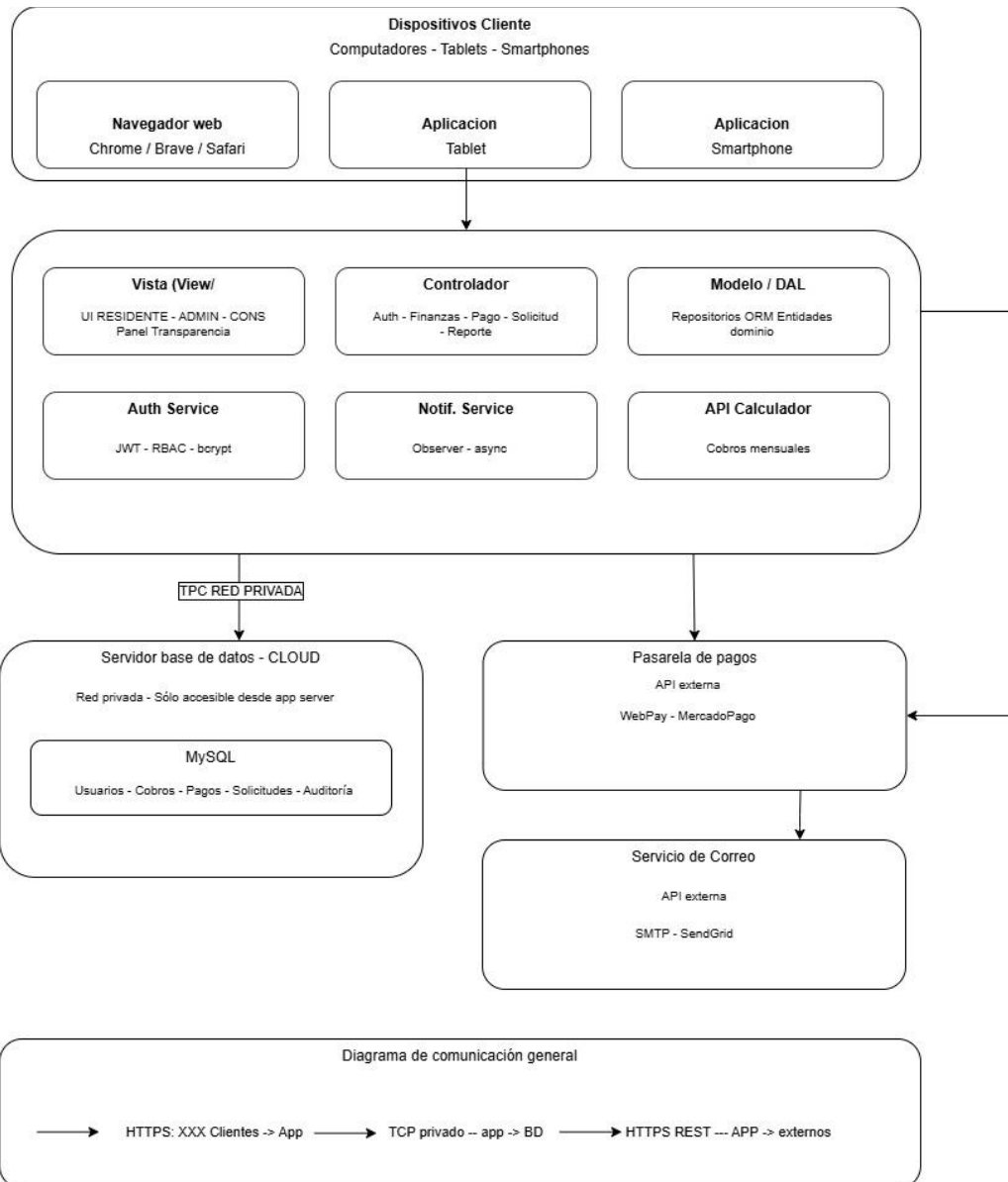
Nodos del despliegue:

- **Dispositivos de Usuario (Clientes):** Computadores, tablets y smartphones. Acceso mediante navegadores web modernos vía HTTPS.

- **Servidor de Aplicación (Cloud — AWS / GCP / Azure):** Aloja la aplicación web monolítica INMOGEST. Ejecuta Nginx/Apache y el runtime de la aplicación. Expone la aplicación mediante HTTPS con certificado SSL/TLS.
- **Servidor de Base de Datos (Cloud):** Nodo separado. Motor relacional PostgreSQL o MySQL. Accesible únicamente desde el Servidor de Aplicación (red privada).
- **Servicios Externos:** Pasarela de Pagos (Transbank/Webpay o MercadoPago) y Servicio de Correo (SMTP/SendGrid), ambos vía HTTPS/REST desde el Servidor de Aplicación.

Comunicaciones:

- Usuario → Servidor de Aplicación: HTTPS (puerto 443)
- Servidor de Aplicación → Base de Datos: TCP privado (puerto 5432/3306, red interna)
- Servidor de Aplicación → Pasarela de Pagos: HTTPS/REST (API externa)
- Servidor de Aplicación → Servicio de Correo: HTTPS/SMTP



5. ESTÁNDARES DE CALIDAD DEL SOFTWARE

5.1 Estándar Aplicado: ISO/IEC 25010 (SQuaRE)

INMOGEST adopta como referencia el estándar internacional **ISO/IEC 25010**, que define el modelo de calidad del producto software mediante características y subcaracterísticas medibles. Este estándar fue seleccionado por su alineación con los fundamentos de la ingeniería de software y por ser el referente internacional más reconocido para la evaluación de la calidad de sistemas.

La justificación de aplicar este estándar en INMOGEST radica en que el sistema maneja **información financiera sensible** (cobros, pagos, datos personales de residentes) y **debe ser utilizado por usuarios no técnicos**, lo que exige altos niveles de seguridad, usabilidad y confiabilidad.

5.2 Características de Calidad Aplicadas al Sistema

- **Funcionalidad (Functional Suitability):** Las funcionalidades están mapeadas a RF-01–RF-20. Se aplicarán pruebas unitarias y de integración para verificar la exactitud de los cálculos financieros.
- **Confiabilidad (Reliability):** Disponibilidad mínima del 99% (RNF-02). Se implementarán mecanismos de recuperación ante fallos y logs de transacciones.
- **Seguridad (Security):** Cifrado HTTPS/TLS, hashing de contraseñas con bcrypt, autenticación JWT, autorización RBAC, y cumplimiento de la Ley 19.628 (RNF-01, RNF-09).
- **Usabilidad (Usability):** Interfaces intuitivas para usuarios no técnicos, navegación clara, mensajes de error comprensibles y diseño responsivo (RNF-03, RNF-04). Se realizarán pruebas con usuarios representativos.
- **Eficiencia en el Desempeño (Performance Efficiency):** Respuesta en menos de 3 segundos bajo condiciones normales (RNF-07). Se optimizarán consultas con índices y paginación.
- **Mantenibilidad (Maintainability):** Arquitectura MVC + Patrón Repositorio para código modular y legible (RNF-08). Estándares de codificación y documentación técnica actualizada.
- **Portabilidad (Portability):** Aplicación web accesible desde cualquier dispositivo con navegador moderno, sin instalación local (RNF-04).
- **Interoperabilidad (Compatibility):** Integración con servicios externos mediante APIs REST y SMTP/HTTPS (RNF-10).

5.3 Métricas de Calidad Definidas

Característica	Métrica	Meta
Disponibilidad	Tiempo de actividad del sistema	>= 99% mensual
Rendimiento	Tiempo de respuesta en operaciones estándar	< 3 segundos
Seguridad	Vulnerabilidades críticas en auditoría	0 vulnerabilidades críticas
Usabilidad	Tasa de errores de navegación con usuarios	< 10% de tareas con error
Mantenibilidad	Cobertura de pruebas unitarias en módulo de finanzas	>= 70%
Confiabilidad	Tasa de éxito en procesamiento de pagos	>= 99.5%

6. CONTROL DE PRUEBAS DE CALIDAD

6.1 Aspectos de Calidad Evaluados

Característica ISO/IEC 25010	Descripción	Criterio de Evaluación
Adecuación Funcional	Verifica que las funcionalidades implementadas satisfacen los requisitos definidos.	Cumplimiento de los casos de uso y requisitos funcionales.
Usabilidad	Evalúa la facilidad de aprendizaje, comprensión y uso del sistema.	Navegación intuitiva y facilidad de interacción.
Fiabilidad	Determina la consistencia del comportamiento del sistema durante la navegación.	Correcto funcionamiento de enlaces y flujos.
Seguridad	Verifica controles básicos de acceso y validación de datos.	Autenticación y validación de formularios.
Compatibilidad	Evalúa la correcta visualización en distintos dispositivos.	Adaptabilidad responsive.

6.2 Casos de prueba aplicados

ID	Funcionalidad	Objetivo de la Prueba	Característica ISO/IEC 25010
CP-01	Registro de Usuario	Verificar que el sistema permita registrar correctamente un nuevo residente validando los datos obligatorios y el rol seleccionado.	Adecuación Funcional, Seguridad
CP-02	Inicio de Sesión	Verificar que únicamente usuarios con credenciales válidas puedan acceder al sistema y que se gestionen correctamente los errores de autenticación.	Seguridad, Adecuación Funcional
CP-03	Consulta de Estado de Cuenta	Verificar la correcta visualización de las deudas, gastos comunes y, cuando corresponda, deuda de arriendo.	Adecuación Funcional, Usabilidad
CP-04	Pago de Deudas	Verificar el flujo completo de pago, incluyendo confirmación de la operación y generación del comprobante correspondiente.	Adecuación Funcional, Fiabilidad
CP-05	Gestión de Solicitudes	Verificar el registro de solicitudes de mantención y la consulta de su estado de seguimiento.	Adecuación Funcional, Usabilidad

CP-06	Registro de Quejas	Verificar que el usuario pueda registrar correctamente una queja o reclamo y visualizar su información posteriormente.	Adecuación Funcional
CP-07	Navegación e Interfaz	Verificar que la navegación entre las distintas pantallas sea consistente, intuitiva y libre de errores.	Usabilidad, Fiabilidad
CP-08	Compatibilidad Responsive	Verificar la correcta visualización y funcionamiento del prototipo en dispositivos de escritorio.	Compatibilidad
CP-09	Gestión del Personal	Verificar que el administrador pueda asignar conserjes, editar sus horarios y consultar correctamente su información de contacto.	Adecuación Funcional
CP-10	Comunicación con Conserjes	Verificar el funcionamiento del chat entre el administrador y los conserjes, permitiendo el envío y recepción de mensajes.	Adecuación Funcional, Usabilidad
CP-11	Gestión Financiera del Inmueble	Verificar que el administrador pueda visualizar correctamente las cuentas, fondos de ahorro y el desglose completo de gastos del condominio.	Adecuación Funcional
CP-12	Envío de Liquidaciones	Verificar que el administrador pueda generar y enviar liquidaciones mediante la funcionalidad disponible en el sistema.	Adecuación Funcional, Fiabilidad

6.3 Formato de Registro de Pruebas

ID	Función del programa	Resultado Esperado	Estado
CP-01	Registro de Usuario	El usuario se registra correctamente validando los datos obligatorios y el rol seleccionado.	Aprobado
CP-02	Inicio de Sesión	El usuario accede correctamente utilizando credenciales válidas.	Aprobado
CP-03	Consulta de Estado de Cuenta	Se visualizan correctamente las deudas y estados asociados.	Aprobado
CP-04	Pago de Deudas	El pago finaliza correctamente y se genera el comprobante correspondiente.	Aprobado
CP-05	Gestión de Solicitudes	Es posible registrar solicitudes de mantención y consultar su estado.	Aprobado
CP-06	Registro de Quejas	Es posible registrar correctamente una queja o reclamo.	Aprobado
CP-07	Navegación e Interfaz	La navegación entre las distintas pantallas es correcta y no presenta errores.	Aprobado
CP-08	Compatibilidad Responsive	El sistema se visualiza correctamente en escritorio y dispositivos móviles.	Aprobado

CP-09	Gestión del Personal	El administrador puede asignar conserjes, modificar horarios y consultar sus datos de contacto.	Aprobado
CP-10	Comunicación con Conserjes	El chat permite una comunicación correcta entre administrador y conserjes.	Aprobado
CP-11	Gestión Financiera del Inmueble	Se visualizan correctamente las cuentas, fondos de ahorro y el desglose de gastos del condominio.	Aprobado
CP-12	Envío de Liquidaciones	El administrador puede generar y enviar liquidaciones correctamente.	Aprobado

6.4 Control de versiones del sistema

Versión	Fecha	Modulo Modificado	Cambios Implementados	Anotaciones	Responsable
1.0	23-jun	Sistema en general	Se desarrolló la primera versión del prototipo de alta fidelidad, incorporando las pantallas y flujos principales orientados al rol de residente: inicio de sesión, visualización del estado de cuenta, consulta de cobros pendientes y opción de pago en línea, estableciendo la base visual y de navegación del sistema.	Corresponde a la primera entrega del prototipo. Sirve como punto de partida para las siguientes iteraciones y aún no incorpora las vistas de administrador ni de conserjería.	Williams Contreras
1.1	25-jun	Sistema en general - Portal de residentes	Se incorporó al portal de residentes un módulo de chat en línea con conserjería, disponible las 24 horas, pensado para resolver consultas rápidas o informales que no requieren el registro formal de una solicitud o incidencia.	El prototipo se actualizó para reflejar visualmente esta nueva función: se agregó el acceso al chat dentro del portal de residentes junto con una vista básica de conversación. Queda pendiente definir el comportamiento del chat fuera del horario de atención.	Horacio Navarrete
1.2	25-jun	Sistema en general - Portal del Administrador	Se diseñó e incorporó al prototipo el portal de administrador, incluyendo las pantallas de gestión de residentes, cobros, morosidad y generación de reportes, complementando la vista de residentes ya existente.	Estas pantallas están orientadas exclusivamente al rol de administrador y consolidan las funciones operativas de control financiero y gestión del edificio dentro del prototipo.	Williams Contreras - Horacio Navarrete

7. ANEXOS

7.1 Tarjetas del Tablero Kanban

Código	Título	Capa	Funcionalidad Asociada
TK01	Diseñar estructura de usuarios, roles y sesiones	Base de Datos	Gestión de usuarios y seguridad
TK02	Implementar modelo de usuarios y autenticación	Backend	Gestión de usuarios y seguridad
TK03	Desarrollar interfaces de registro, login y perfiles	Frontend	Gestión de usuarios y seguridad
TK04	Implementar controladores de acceso y permisos	Backend	Gestión de usuarios y seguridad
TK05	Configurar seguridad y auditoría de usuarios	Infraestructura	Gestión de usuarios y seguridad
TK06	Diseñar dashboards para residentes, administradores y conserjes	Frontend	Gestión operativa y dashboards
TK07	Implementar lógica de gestión operativa	Backend	Gestión operativa y dashboards
TK08	Implementar controladores de operaciones de usuarios	Backend	Gestión operativa y dashboards
TK09	Configurar registro de eventos operativos	Infraestructura	Gestión operativa y dashboards
TK10	Diseñar estructura financiera de gastos, cobros y pagos	Base de Datos	Módulo financiero
TK11	Implementar lógica de cálculo de gastos comunes	Backend	Módulo financiero
TK12	Implementar emisión y registro de cobros y pagos	General	Módulo financiero
TK13	Diseñar vistas financieras y comprobantes	Frontend	Módulo financiero
TK14	Configurar notificaciones financieras	Infraestructura	Módulo financiero
TK15	Diseñar estructura de solicitudes, reservas y notificaciones	Infraestructura	Solicitudes, reservas y notificaciones
TK16	Implementar lógica de solicitudes y reservas	Backend	Solicitudes, reservas y notificaciones
TK17	Diseñar interfaces de solicitudes, reservas y notificaciones	Frontend	Solicitudes, reservas y notificaciones
TK18	Implementar controladores de flujo y seguimiento	Backend	Solicitudes, reservas y notificaciones
TK19	Configurar motor de notificaciones automáticas	Infraestructura	Solicitudes, reservas y notificaciones
TK20	Diseñar estructuras de proveedores, mantenimientos y documentos	Base de Datos	Proveedores y mantenimiento
TK21	Implementar modelos de proveedores y mantenimiento	Backend	Proveedores y mantenimiento

TK22	Diseñar interfaces administrativas y documentales	Frontend	Proveedores y mantenimiento
TK23	Implementar controladores de gestión operativa	Backend	Proveedores y mantenimiento
TK24	Configurar validaciones y seguridad documental	Infraestructura	Proveedores y mantenimiento
TK25	Diseñar estructura de reportes e historial financiero	Base de Datos	Reportes e incidencias
TK26	Implementar generación de reportes y trazabilidad	General	Reportes e incidencias
TK27	Diseñar interfaces de reportes e incidencias	Frontend	Reportes e incidencias
TK28	Implementar controladores de incidencias y exportación	Backend	Reportes e incidencias
TK29	Configurar exportaciones y notificaciones de cierre	Infraestructura	Reportes e incidencias

7.2 Tablero Kanban

